PANTHORA CLOTHING — E-COMMERCE PROJECT

Complete Technical Documentation & Beginner's Presentation Guide

1. Executive Summary & Technology Stack

Panthora Clothing is a full-stack modern streetwear e-commerce web application built using the **MERN Stack** (MongoDB, Express.js, React.js, Node.js) with Tailwind CSS for styling and Redux Toolkit for shopping cart state management. The project provides a fast, sleek shopping experience for streetwear apparel including Hoodies, Oversized Tees, and Trackpants.

Technology Layer	Tool / Library	Purpose / Description
Frontend UI	React.js 18 + Vite	Builds interactive single-page user interface with super-fast loading.
Styling & CSS	Tailwind CSS	Utility-first CSS framework for modern dark-theme streetwear design.
State Management	Redux Toolkit (@reduxjs/toolkit)	Manages shopping cart state across the entire application globally.
Backend API	Node.js + Express.js	Handles web server requests, route logic, and API data delivery.
Database	MongoDB + Mongoose (with Fallback)	Stores products, user profiles, and customer orders permanently.
Image Assets	Static Express / Vite Public Uploads	Serves actual product photos directly from local uploads folder.

2. System Architecture & Request Flow

Beginner Primer: How does the application run?
When a user opens the website in a web browser (at http://localhost:5173), the **React Frontend** loads. The React app sends an HTTP API request (using Axios) to the **Express Backend Server** running at http://localhost:5000/api/products. The Backend checks the database (MongoDB or In-Memory Store), fetches product details with local image paths (e.g., /uploads/black-hoodie.png), and sends back JSON data to React, which renders product cards dynamically on screen.

Step-by-Step Request Flow:

1. **Browser Request:** User visits Home Page or Category Page.
2. **React API Call:** Frontend executes axios.get('/api/products?category=hoodie').
3. **Express Route:** Express matches route GET /api/products in productRoutes.js.
4. **Controller Execution:** productController.js queries MongoDB / MemoryStore.
5. **Image & JSON Response:** Backend returns JSON array of products with image paths for React to render.

3. Complete Project Folder Structure Explained

Folder / File Path	Role & Significance
Clothes photos/	Source directory where user uploads high-quality clothing images.
client/src/pages/	Contains main pages: **Home.jsx**, **Products.jsx**, **ProductDetail.jsx**, **Cart.jsx**, **Checkout.jsx**.
client/src/components/	Reusable UI components like **Navbar.jsx**, **Footer.jsx**, **ProductCard.jsx**, **CartItem.jsx**.
client/src/store/	Redux Toolkit setup (store.js, cartSlice.js, authSlice.js) for state management.
client/public/uploads/	Static frontend folder containing normalized clothing images served directly by Vite.
server/server.js	Main entry point for Express server; mounts routes and static upload folder.
server/store.js	In-memory database store with initial product array fallback.
server/seed.js	Script to populate database with default admin user and product catalog.
server/config/db.js	MongoDB connection handler with automatic fallback and auto-seeding.
server/controllers/	Business logic functions for authentication, product management, and order processing.

4. Key Backend Code Explanation (Line-by-Line)

File 1: server/server.js (Express Server Entry Point)

What it does: Initializes Node.js Express server, enables CORS security headers, mounts static path for uploaded images, and registers API endpoints.

```
const express = require('express');
const cors = require('cors');
const path = require('path');
const connectDB = require('./config/db');

const app = express();
app.use(cors({ origin: 'http://localhost:5173', credentials: true }));
app.use(express.json());
// SERVE CLOTHING PHOTOS STATICALLY:
app.use('/uploads', express.static(path.join(__dirname, 'uploads')));

// MOUNT ROUTES:
app.use('/api/products', require('./routes/productRoutes'));
app.use('/api/auth', require('./routes/authRoutes'));
app.use('/api/orders', require('./routes/orderRoutes'));

app.listen(5000, () => console.log('Panthora API running on port 5000'));
connectDB();
```

File 2: server/controllers/productController.js (Product API Logic)

What it does: Contains controller logic to fetch products filtered by category (hoodie, tee, lower), sort by price, and return single product details.

```
const getProducts = async (req, res) => {
  const { category, sort, featured } = req.query;
  const filter = {};
  if (category) filter.category = category; // Filter by hoodie, tee, lower
  if (featured === 'true') filter.isFeatured = true;

  let query = Product.find(filter);
  if (sort === 'price_asc') query = query.sort({ price: 1 });
  else if (sort === 'price_desc') query = query.sort({ price: -1 });
  else query = query.sort({ createdAt: -1 });

  const products = await query;
  res.json({ success: true, count: products.length, products });
};
```

5. Key Frontend Code Explanation (React & Components)

File 1: client/src/components/ProductCard.jsx (Displaying Clothing Item)

What it does: A reusable UI component that renders each clothing product card with image, title, price, category tag, and Quick Add to Cart button.

```
export default function ProductCard({ product }) {
  const dispatch = useDispatch();
  return (
    <div className="card group relative overflow-hidden">
      {/* Product Image */}
      <img src={product.images?.[0]} alt={product.name} className="w-full h-80 object-cover" />
      <div className="p-4">
        <span className="text-xs text-accent uppercase">{product.category}</span>
        <h3 className="font-bold text-lg">{product.name}</h3>
        <p className="text-accent font-semibold">Rs. {product.price}</p>
        <button onClick={() => dispatch(addToCart(product))} className="btn-primary w-full mt-3">
          Add to Cart
        </button>
      </div>
    </div>
  );
}
```

6. Category & Image Mapping Overview

All old stock images were removed and replaced with actual uploaded clothing photos. Images are normalized in client/public/uploads/ and server/uploads/ as shown below:

Category	Sample Product Name	Image File Name	Preview
Hoodies	Black Oversized Hoodie	black-hoodie.png	
Tees	Printed Oversized Tee	printed-oversized-tee.png	
Lowers	Black Trackpants	black-trackpants.png	

7. Presentation Guide & Common Questions From Teachers

When presenting this project to your sir/teacher, use these simple explanations to explain your project confidently:

Q1: What is the main objective of this project?

Ans: Panthora Clothing is a streetwear e-commerce web app allowing users to browse apparel by category (Hoodies, Tees, Trackpants), view product details, add items to a cart, and place orders.

Q2: Why did you choose the MERN Stack?

Ans: React.js provides a single-page application experience with fast page loading. Node.js and Express handle backend APIs, and MongoDB provides flexible data storage for products and orders.

Q3: How are product images stored and served?

Ans: Images are stored in local upload directories (/uploads). Express serves them statically via app.use('/uploads', express.static(...)) so both frontend and backend load photos directly.

Q4: How does cart state management work?

Ans: Shopping cart state is managed using Redux Toolkit (@reduxjs/toolkit). Items added to cart are stored in Redux state and synced with browser localStorage.